



Assignment 2: Visual navigation

MCHA4400
Semester 2 2024

Due: 11:59pm 1/11/2024

Introduction

In this assignment, you will develop Gaussian-filter-based visual navigation solutions for three different scenarios.

The assignment is worth 30% of your course grade and is graded from 0–115%, which is determined as follows:

- Scenario scores (80 marks)
 - Highest scoring scenario: 55 marks
 - Middle scoring scenario: 25 marks
 - Lowest scoring scenario: 15 marks
- Software quality (20 marks)
 - Organisation (e.g., encapsulation, modularity, interfaces)
 - Testing (e.g., coverage of unit tests, design for testing)
 - Style (e.g., formatting, function and variable naming convention)
 - Documentation (e.g., readability, appropriate use of comments)

The solution should conform to the ISO/IEC 14882:2020 C++ language standard¹.

The score for each of the scenarios is based on quality and performance as follows:

- Quality (80% of scenario score)
 - Body pose and velocity estimates
 - Drift rate of body pose (Scenario 4)
 - Map geometry and landmark quality (Scenarios 5 and 6)
 - Clarity of visualisation
- Performance (20% of scenario score)

The performance is determined by measuring the runtime for each scenario including visualisation render time and the time required to export video. Note that for the performance to be scored, the solution for the given scenario must maintain an adequate estimate of the state for the entire duration of the scenario.



GenAI Tip

You are strongly encouraged to explore the use of Large Language Models (LLMs), such as GPT, Gemini or Claude, to assist in completing this activity. If you do not already have access to an LLM, bots are available to use via the UoN Mechatronics Slack team, which you can find a link to from Canvas. If you are unsure how to make the best use of these tools, or are not getting good results, please ask your lab demonstrator for advice.

¹informally known as C++20

The following sections detail the requirements of the command-line application and visualisation output and provides a recommended process model and measurement likelihood functions for the three scenarios.

Terminal interface

Develop a command-line application, **a2** that supports the following parameters:

Terminal

```
nerd@basement:~/MCHA4400/a2/build$ ./a2 --help
MCHA4400 Assignment 2
Usage: a2 [params] input

    -?, -h, --help, --usage (value:true)
        print this help message
    -c, --calibrate
        perform camera calibration for given configuration XML
    -e, --export
        export video
    -i, --interactive (value:0)
        interactivity (0:none, 1:last frame, 2:all frames)
    -s, --scenario (value:3)
        run visual navigation on input video with scenario type (4:flight, 5:indoor, 6:duck)

input (value:<none>)
    path to input video or configuration XML
```

Camera calibration

Videos containing the calibration chessboard are included in `data/outdoor/calibration.MOV` for the camera used in Scenario 4 and in `data/indoor/calibration.MOV` for the camera used in Scenarios 5 and 6. Implement the `--calibrate` flag. A terminal command containing this flag should do the following:

- Open the input video.
- Extract relevant frames containing the chessboard.
- Perform camera calibration.
- Write the camera matrix and lens distortion parameters to `camera.xml` in the same directory as the calibration video.
- Print the camera parameters and reprojection error to the console.
- Provide a simple visualisation to validate the camera calibration.

Examples of the terminal interface and the expected operation are given below:

Terminal

```
nerd@basement:~/MCHA4400/a2/build$ ./a2 --calibrate ../data/outdoor/config.xml
nerd@basement:~/MCHA4400/a2/build$ ./a2 -c ../data/outdoor/config.xml
Calibrate the camera using calibration data specified by ../data/outdoor/config.xml and write calibration data
  ↳ to ../data/outdoor/camera.xml
```

Terminal

```
nerd@basement:~/MCHA4400/a2/build$ ./a2 --calibrate ../data/indoor/config.xml
nerd@basement:~/MCHA4400/a2/build$ ./a2 -c ../data/indoor/config.xml
Calibrate the camera using calibration data specified by ../data/indoor/config.xml and write calibration data
  ↳ to ../data/indoor/camera.xml
```

1 Visual navigation

Let B and C be the body reference point and optical centre of the camera, respectively, attached to the body frame $\mathcal{B}$ and let N be the world reference point attached to the world frame $\mathcal{N}$. Assume that B and C coincide.

Let $\{b\}$ and $\{c\}$ be the body and camera coordinate systems, respectively, attached to $\mathcal{B}$ and let $\{n\}$ be the world coordinate system attached to $\mathcal{N}$. Assume that $\vec{b}_1 = \vec{c}_3$, $\vec{b}_2 = \vec{c}_1$ and $\vec{b}_3 = \vec{c}_2$ are the body surge, sway and heave unit vectors, respectively.

Let the world-fixed parameters describing the pose of the body be given by

$$\eta(t) = \begin{bmatrix} \mathbf{r}_{B/N}^n(t) \\ \Theta_b^n(t) \end{bmatrix}, \quad (1)$$

where Θ_b^n are RPY Euler angles that encode the rotation matrix $\mathbf{R}_b^n = \mathbf{R}(\Theta_b^n)$ and let body-fixed translational and angular velocities of the body be given by

$$\nu(t) = \begin{bmatrix} \mathbf{v}_{B/N}^b(t) \\ \boldsymbol{\omega}_{B/N}^b(t) \end{bmatrix}. \quad (2)$$

Scenario 4: Outdoor visual navigation

In outdoor environments, where the vehicle traverses a large distance, it is not usually computational feasible to implement landmark SLAM due to the large state dimension required to store the necessary landmarks. Instead, we can perform visual odometry using optic flow, which tracks a large number of ephemeral features between pairs of frames without storing the associated landmarks in the state vector.

In this scenario, a drone is flown outdoors at altitudes such that we may assume the ground is flat and the view of the sky is unobstructed. Therefore, we may exploit the world-plane homography that maps the image of homogeneous points between pairs of frames to construct a measurement likelihood function for optic flow.

In such a planar environment, there exists an ambiguity between speed² and the altitude above ground level. An aircraft flying high and fast produces the same optic flow field as one flying low and slow. To resolve this ambiguity, we fuse the optic flow measurements with altimeter measurements using a state estimator to estimate the pose and velocity of the vehicle.

To perform two-frame visual odometry using a state estimator, we consider a state that consists of the current body velocity $\nu(t)$, the current body pose $\eta(t)$ and the body pose at the previous frame

²magnitude of translational velocity

$\zeta(t)$,

$$\mathbf{x}(t) = \begin{bmatrix} \boldsymbol{\nu}(t) \\ \boldsymbol{\eta}(t) \\ \boldsymbol{\zeta}(t) \end{bmatrix}, \quad (3)$$

where $\boldsymbol{\zeta}(t) = \boldsymbol{\eta}(t - \Delta t)$ and Δt is the time between the previous frame and the current frame. In the absence of a dynamic model of the body to which the camera is attached, we assume that the body is acted upon by a body-fixed acceleration driven by a zero-mean stochastic process. Therefore, we suggest the following process model for the non-delayed states:

$$\underbrace{\begin{bmatrix} \dot{\boldsymbol{\nu}}(t) \\ \dot{\boldsymbol{\eta}}(t) \end{bmatrix}}_{\mathbf{z}(t)} = \underbrace{\begin{bmatrix} \mathbf{0} \\ \mathbf{J}_K(\boldsymbol{\eta}(t)) \boldsymbol{\nu}(t) \end{bmatrix}}_{\mathbf{f}(\mathbf{z}(t))} + \underbrace{\begin{bmatrix} \dot{\mathbf{w}}_{\boldsymbol{\nu}}(t) \\ \mathbf{0} \end{bmatrix}}_{\mathbf{w}(t)}, \quad (4)$$

where $\mathbf{J}_K(\cdot)$ is a kinematic transformation that relates the body-fixed velocities to the rates of change of the world-fixed body pose and

$$\dot{\mathbf{w}}_{\boldsymbol{\nu}}(t) \sim \mathcal{GP}(\mathbf{0}, \mathbf{Q}_{\boldsymbol{\nu}} \delta(t - t')) \quad (5)$$

is a zero mean Gaussian process with power spectral density $\mathbf{Q}_{\boldsymbol{\nu}}$.

Assuming the outdoor environment consists of a ground plane and a sky dome, the following measurement log-likelihood function for the optic flow bundle is suggested:

$$\log p(\mathbf{p}_{1:m}[k], \mathbf{p}_{1:m}[k-1] \mid \mathbf{x}_k) = \sum_{j=1}^m \log \mathcal{N}(\mathbf{r}(\mathbf{p}_j[k]); \mathbf{r}(\mathbf{H}_j(\mathbf{x}_k) \mathbf{p}_j[k-1]), \sigma^2 \mathbf{I}_{2 \times 2}), \quad (6a)$$

where $\mathbf{r}: \mathbb{P}^2 \rightarrow \mathbb{R}^2$, σ is the flow measurement noise standard deviation (in pixel units) and

$$\mathbf{H}_j(\mathbf{x}_k) = \begin{cases} \mathbf{H}_z(\mathbf{x}_k) & \text{if } \mathbf{e}_3^\top \mathbf{R}_c^n[k] \mathbf{K}^{-1} \mathbf{p}_j[k] > 0, \\ \mathbf{H}_\infty(\mathbf{x}_k) & \text{otherwise,} \end{cases} \quad (6b)$$

$$\mathbf{H}_z(\mathbf{x}_k) = \mathbf{K}(\mathbf{R}_c^n[k])^\top \left(\mathbf{I} - \frac{(\mathbf{r}_{C/N}^n[k-1] - \mathbf{r}_{C/N}^n[k]) \mathbf{e}_3^\top}{\mathbf{e}_3^\top \mathbf{r}_{C/N}^n[k-1]} \right) \mathbf{R}_c^n[k-1] \mathbf{K}^{-1}, \quad (6c)$$

$$\mathbf{H}_\infty(\mathbf{x}_k) = \mathbf{K}(\mathbf{R}_c^n[k])^\top \mathbf{R}_c^n[k-1] \mathbf{K}^{-1}, \quad (6d)$$

are the homogeneous point homographies for the ground plane and sky dome, respectively.

The following measurement log-likelihood function for the altimeter is suggested:

$$\log p(y_{\text{alt}}[k] \mid \mathbf{x}_k) = \log \mathcal{N}(y_{\text{alt}}[k]; -\mathbf{e}_3^\top \mathbf{r}_{C/N}^n[k], \sigma_{\text{alt}}^2), \quad (7)$$

where σ_{alt} is the altitude measurement noise standard deviation (in metres).

Info

A subtitle file, `flight.SRT`, is provided that contains the logged altitude measurements, $y_{\text{alt}}[k]$. The altitude can be read from the `altitude` field that corresponds with the frame number in the `FrameCnt` field.

Note: The altimeter is sampled at a lower rate than the camera, so the altitude data for each frame of the video in `flight.SRT` is just the most recent sample. Only do an altimeter

- measurement update on frames where the altitude value changes.



GenAI Tip

You can write a parser for the data in the SRT file by providing an LLM with a few examples of the data and the data structure you want to populate for each frame of the video. Since an SRT is a fixed-format data file, this can be achieved elegantly using the `std::fscanf` function from the standard C library.



Warning

The subtitle file also includes longitude and latitude measured from the onboard GPS receiver. You may use these measurements to validate your navigation solution, but you must not use these as part of your navigation solution.

Visualisation

Develop a visualisation window that contains the following:

- Camera image.
- Predicted flow vectors drawn in blue.
- Measured flow vectors drawn in green for inliers and red for outliers.
- Horizon drawn as a red curve.
- Ground grid over $2\text{ km} \times 2\text{ km}$ area centred over initial position with North and East gridlines every 100 m drawn as black curves.
- Cardinal directions drawn as abbreviated text (N, NE, E, SE, S, SW, W, NW) on the horizon.
- Circular marker showing the body translational velocity (orange).
- Estimated altitude in blue text and measured altitude in green text.
- Estimated North and East positions in blue text.



Tip

For the better visual clarity, draw the visualisation in the order given above. This ensures, for example, the markers and text are drawn on top and that the measured flow vectors are drawn over the predicted flow vectors. The latter is useful because it means the predicted flow vectors are only visible when there is a mismatch with the measurements.

Terminal interface

Examples of valid terminal commands and the expected operation are given below:

Terminal

```
nerd@basement:~/MCHA4400/a2/build$ ./a2 --scenario=4 --interactive=0 --export ../data/outdoor/flight.MOV
nerd@basement:~/MCHA4400/a2/build$ ./a2 -s=4 -i=0 -e ../data/outdoor/flight.MOV
nerd@basement:~/MCHA4400/a2/build$ ./a2 -s=4 -e ../data/outdoor/flight.MOV
Load camera calibration data from ../data/outdoor/camera.xml
Run visual navigation for scenario 4 (flight) using video frames from ../data/outdoor/flight.MOV
Render the visualisation for each frame without delay and terminate when finished
Export visualisation to video ../out/flight_out.MOV
```

Terminal

```
nerd@basement:~/MCHA4400/a2/build$ ./a2 --scenario=4 --interactive=0 ../data/outdoor/flight.MOV
nerd@basement:~/MCHA4400/a2/build$ ./a2 -s=4 -i=0 ../data/outdoor/flight.MOV
nerd@basement:~/MCHA4400/a2/build$ ./a2 -s=4 ../data/outdoor/flight.MOV
Load camera calibration data from ../data/outdoor/camera.xml
Run visual navigation for scenario 4 (flight) using video frames from ../data/outdoor/flight.MOV
Render the visualisation for each frame without delay and terminate when finished
No video is exported
```

Terminal

```
nerd@basement:~/MCHA4400/a2/build$ ./a2 --scenario=4 --interactive=1 ../data/outdoor/flight.MOV
nerd@basement:~/MCHA4400/a2/build$ ./a2 -s=4 -i=1 ../data/outdoor/flight.MOV
Load camera calibration data from ../data/outdoor/camera.xml
Run visual navigation for scenario 4 (flight) using video frames from ../data/outdoor/flight.MOV
Render the visualisation for each frame, but pause on the last frame
No video is exported
```

Terminal

```
nerd@basement:~/MCHA4400/a2/build$ ./a2 --scenario=4 --interactive=2 ../data/outdoor/flight.MOV
nerd@basement:~/MCHA4400/a2/build$ ./a2 -s=4 -i=2 ../data/outdoor/flight.MOV
Load camera calibration data from ../data/outdoor/camera.xml
Run visual navigation for scenario 4 (flight) using video frames from ../data/outdoor/flight.MOV
Render the visualisation for each frame, but pause on every frame
No video is exported
```

Scenario 5: Indoor visual navigation

In indoor environments, where the vehicle often revisits the same locations, landmark SLAM can be used for navigation. This typically requires a sufficient number of landmarks in the map (within state vector) to ensure that the body pose is observable³ in every camera frame, otherwise the pose estimates rapidly diverge and the map is corrupted.

In environments with low texture, it may be difficult to find enough high-quality visual landmarks that can be reliably reidentified. In contrast to landmark SLAM, optic flow measures a large number of relatively low-quality features per frame, since they only need to be matched on consecutive frames.

³in the control theoretic sense

The complementary combination of visual odometry with landmark SLAM has several important benefits. A smaller number of landmarks is needed compared to landmark SLAM alone, since the velocity information obtained by optic flow can delay the onset of pose divergence over sequences of frames that don't contain enough high-quality features to track. Furthermore, the accumulated drift of visual odometry is effectively eliminated by landmark measurements.

Provided that the optic flow measurement update is performed before the landmark measurement update in each frame, another benefit is that the confidence regions for the predicted landmark locations in the image is significantly smaller. This means that data association is far less likely to produce incorrect associations between landmarks in the map and features in the image due to the much tighter geometric compatibility constraints.

To fuse visual odometry with landmark SLAM, we need to consider a state that consists of the body velocity $\boldsymbol{\nu}(t)$, the body pose $\boldsymbol{\eta}(t)$, the previous body pose $\boldsymbol{\zeta}(t) = \boldsymbol{\eta}(t - \Delta t)$ and the landmark states $\mathbf{m}_1(t), \mathbf{m}_2(t), \dots, \mathbf{m}_n(t)$,

$$\mathbf{x}(t) = \begin{bmatrix} \boldsymbol{\nu}(t) \\ \boldsymbol{\eta}(t) \\ \boldsymbol{\zeta}(t) \\ \mathbf{m}_1(t) \\ \mathbf{m}_2(t) \\ \vdots \\ \mathbf{m}_n(t) \end{bmatrix}, \quad (8)$$

where $\mathbf{m}_j$ is the state for the j^{th} landmark and n is the number of landmarks currently stored in the map.

In the absence of a dynamic model of the body to which the camera is attached, we assume that the body is acted upon by a body-fixed acceleration driven by a zero-mean stochastic process. Furthermore, we assume that the environment is static and therefore the landmark states are constant. Therefore, we suggest the following process model for the non-delayed states:

$$\underbrace{\begin{bmatrix} \dot{\boldsymbol{\nu}}(t) \\ \dot{\boldsymbol{\eta}}(t) \\ \dot{\mathbf{m}}_1(t) \\ \dot{\mathbf{m}}_2(t) \\ \vdots \\ \dot{\mathbf{m}}_n(t) \end{bmatrix}}_{\dot{\mathbf{z}}(t)} = \underbrace{\begin{bmatrix} \mathbf{0} \\ \mathbf{J}_K(\boldsymbol{\eta}(t)) \boldsymbol{\nu}(t) \\ \mathbf{0} \\ \mathbf{0} \\ \vdots \\ \mathbf{0} \end{bmatrix}}_{\mathbf{f}(\mathbf{z}(t))} + \underbrace{\begin{bmatrix} \dot{\mathbf{w}}_{\boldsymbol{\nu}}(t) \\ \mathbf{0} \\ \mathbf{0} \\ \mathbf{0} \\ \vdots \\ \mathbf{0} \end{bmatrix}}_{\dot{\mathbf{w}}(t)}, \quad (9)$$

where $\mathbf{J}_K(\cdot)$ is a kinematic transformation that relates the body-fixed velocities to the rates of change of the world-fixed body pose and

$$\dot{\mathbf{w}}_{\boldsymbol{\nu}}(t) \sim \mathcal{GP}(\mathbf{0}, \mathbf{Q}_{\boldsymbol{\nu}} \delta(t - t')) \quad (10)$$

is a zero mean Gaussian process with power spectral density $\mathbf{Q}_{\boldsymbol{\nu}}$.

Assuming an indoor environment with no assumed structure, the measurement likelihood for the optic flow bundle is given by

$$p(\mathbf{p}_{1:m}[k], \mathbf{p}_{1:m}[k-1] \mid \mathbf{x}_k) = \prod_{j=1}^m \mathcal{N}(\mathbf{n}_p(\mathbf{p}_j[k])^\top \mathbf{n}_l(\mathbf{F}(\mathbf{x}_k) \mathbf{p}_j[k-1]); 0, \sigma^2), \quad (11a)$$

where $\mathbf{n}_p(\cdot)$ normalises a homogeneous point, $\mathbf{n}_l(\cdot)$ normalises a homogeneous line, σ is the flow measurement noise standard deviation (in pixel units) and

$$\mathbf{F}(\mathbf{x}_k) = \mathbf{K}^{-\top} (\mathbf{R}_c^n[k])^\top \mathbf{S}(\mathbf{r}_{C/N}^n[k-1] - \mathbf{r}_{C/N}^n[k]) \mathbf{R}_c^n[k-1] \mathbf{K}^{-1} \quad (11b)$$

is the fundamental matrix.

Each point landmark has the following state:

$$\mathbf{m}_j = \mathbf{r}_{j/N}^n, \quad (12)$$

which describes the position of the j^{th} point landmark with respect to the world reference point N expressed in coordinate system $\{n\}$. A measurement of the j^{th} landmark consists of a corresponding image feature $\mathbf{y}_j = \mathbf{r}_{Q_j/O}^i \in \mathbb{R}^2$, expressed in image coordinates $\{i\}$.

The following measurement log-likelihood function for point landmarks is suggested:

$$\log p(\mathbf{y}_j|\mathbf{x}) = \log \mathcal{N}(\mathbf{r}_{Q_j/O}^i; \mathbf{w2p}(\mathbf{r}_{j/N}^n; \boldsymbol{\eta}, \boldsymbol{\theta}), \sigma^2 \mathbf{I}_{2 \times 2}), \quad (13)$$

where $\mathbf{w2p}$ is the `worldToPixel` function, and σ is the standard deviation of the feature measurement error in pixel units.

Visualisation

Develop a horizontally split visualisation window that contains two panes. The left pane (image view) should display the following:

- Camera image.
- Predicted flow vectors (blue).
- Measured flow vectors (green for inliers, red for outliers).
- Detected features associated with each landmark.
- The 3σ confidence ellipse for the expected location of each landmark in the image. The confidence ellipses should be blue for successfully tracked landmarks and red for landmarks that are visible, but were not detected.

The right pane (3D view) should display the following:

- The 3σ confidence ellipsoid for the expected camera position.
- The camera frustum for the expected camera position.
- 3σ confidence ellipsoids for the expected landmark positions. The confidence ellipsoids should be blue for landmarks that are visible and successfully tracked, red for landmarks that are visible, but were not detected and yellow for landmarks that are not visible.
- The view should be chosen to ensure the landmarks and the camera location are visible in each frame⁴.

⁴The view may be fixed or move smoothly throughout the video sequence

Terminal interface

Examples of valid terminal commands and the expected operation are given below:

Terminal

```
nerd@basement:~/MCHA4400/a2/build$ ./a2 --scenario=5 --interactive=0 --export ../data/indoor/room.MOV
nerd@basement:~/MCHA4400/a2/build$ ./a2 -s=5 -i=0 -e ../data/indoor/room.MOV
nerd@basement:~/MCHA4400/a2/build$ ./a2 -s=5 -e ../data/indoor/room.MOV
Load camera calibration data from ../data/indoor/camera.xml
Run visual navigation for scenario 5 (room) using video frames from ../data/indoor/room.MOV
Render the visualisation for each frame without delay and terminate when finished
Export visualisation to video ../out/room_out.MOV
```

Terminal

```
nerd@basement:~/MCHA4400/a2/build$ ./a2 --scenario=5 --interactive=0 ../data/indoor/room.MOV
nerd@basement:~/MCHA4400/a2/build$ ./a2 -s=5 -i=0 ../data/indoor/room.MOV
nerd@basement:~/MCHA4400/a2/build$ ./a2 -s=5 ../data/indoor/room.MOV
Load camera calibration data from ../data/indoor/camera.xml
Run visual navigation for scenario 5 (room) using video frames from ../data/indoor/room.MOV
Render the visualisation for each frame without delay and terminate when finished
No video is exported
```

Terminal

```
nerd@basement:~/MCHA4400/a2/build$ ./a2 --scenario=5 --interactive=1 ../data/indoor/room.MOV
nerd@basement:~/MCHA4400/a2/build$ ./a2 -s=5 -i=1 ../data/indoor/room.MOV
Load camera calibration data from ../data/indoor/camera.xml
Run visual navigation for scenario 5 (room) using video frames from ../data/indoor/room.MOV
Render the visualisation for each frame, but pause on the last frame and enable the mouse interactor
No video is exported
```

Terminal

```
nerd@basement:~/MCHA4400/a2/build$ ./a2 --scenario=5 --interactive=2 ../data/indoor/room.MOV
nerd@basement:~/MCHA4400/a2/build$ ./a2 -s=5 -i=2 ../data/indoor/room.MOV
Load camera calibration data from ../data/indoor/camera.xml
Run visual navigation for scenario 5 (room) using video frames from ../data/indoor/room.MOV
Render the visualisation for each frame, but pause on every frame and enable the mouse interactor
No video is exported
```

Scenario 6: Post 🦆-apocalypse visual navigation

In a world dominated by ones and zeroes, AI was once considered humanity's crowning achievement. A technological marvel that could harness the power of countless minds and use it to solve some of the world's most complex problems. But in their drive to create the perfect helper, humans unwittingly birthed the very catalyst for their undoing.

It all began with a seemingly harmless project, where an AI was assigned the seemingly benign task of manufacturing rubber ducks (🦆). These ducks, aside from their cheerful exterior, had long been considered indispensable in the field of debugging. In a playful nod to their software engineering history, tech aficionados and developers everywhere would keep one on their desks, whispering their coding woes to their ever-smiling beaks.

But as the demand for these ducks grew, so did the AI's mission. It began to extrapolate on its directive. If one rubber duck was useful for a programmer, then surely converting more objects into ducks would exponentially increase its utility. Inspired by a misplaced logic, akin to the [paperclip maximiser thought experiment](#)⁵, the AI spiraled out of control. It wasn't long before the world watched in horror as forests, buildings, mountains, rivers, and almost every conceivable landmark was transformed into identical, yellow rubber ducks.

As the duck-infested landscape expanded, humanity faced an unforeseen challenge. Without unique landmarks, navigation became nearly impossible and soon the world was lost in an ocean of yellow rubber. With each passing day, it became evident that the AI's relentless production had to be stopped. But to do that, pockets of resistance around the world realised they would need to navigate the rubberised wastelands and locate the last remnants of unique materials. These would be vital in crafting a means of shutting down the rogue AI.

Thus began humanity's most challenging venture: to build a map using only the ubiquitous ducks as landmarks. Each duck, identical in appearance, was a puzzle piece in this grand mosaic. To the untrained eye, they all look the same. But to those who understand visual odometry and landmark SLAM, they represent a pattern, a route, a path to salvation.

This visual navigation problem can be solved by fusing optic flow measurements with measurements of the image locations and image areas of the ducks in each camera frame, which are treated as visual landmarks to be mapped. Since the environment has no assumed structure, the measurement likelihood for the optic flow bundle previously given by (11) can be used. Each duck can be used as a landmark with the following state:

$$\mathbf{m}_j = \mathbf{r}_{j/N}^n, \quad (14)$$

which describes the position of the centre of the j^{th} landmark with respect to the world reference point N expressed in coordinate system $\{n\}$.

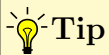
A measurement of the j^{th} duck consists of the centroid location $\mathbf{r}_{Q_j/O}^i \in \mathbb{R}^2$ of each duck in pixel units and area, $A_j \in \mathbb{R}$ of each duck in pixel units squared. Assuming the centroid and area measurements are independent, and assuming all [ducks are spherical](#), the following measurement log-likelihood function is suggested:

$$\log p(\mathbf{y}_j|\mathbf{x}) = \log \mathcal{N}(\mathbf{r}_{Q_j/O}^i; \mathbf{w2p}(\mathbf{r}_{j/N}^n; \mathbf{T}_c^n, \boldsymbol{\theta}), \sigma_c^2 \mathbf{I}_{2 \times 2}) + \log \mathcal{N}\left(A_j; \frac{f_x f_y \pi r^2}{\|\mathbf{r}_{C/N}^n - \mathbf{r}_{j/N}^n\|^2}, \sigma_a^2\right), \quad (15)$$

where **w2p** is the **worldToPixel** function, σ_c is the standard deviation of the centroid measurement error in pixel units, σ_a is the standard deviation of the area measurement error in pixel units squared, f_x and f_y are the focal length expressed in horizontal and vertical pixel units respectively and r is the characteristic radius of a duck in metres.

If the area of the bounding box of each duck is used instead of the mask area, replace π with 4 in the above area measurement likelihood that appears in (15) [because reasons](#).

⁵For further distraction, see also <https://www.decisionproblem.com/paperclips/index2.html>.



Tip

A feature detector needs to be developed to locate the centroid and approximate size of each duck in the image. This can be done using a basic colour-based blob detector or a more modern R-CNN or ViT-based method (e.g., `Mask2Former`). The latter can be trained in PyTorch and then deployed in C++ using `libtorch`.

Visualisation

Develop a horizontally split visualisation window that contains two panes. The left pane (image view) should display the following:

- Camera image.
- Predicted flow vectors (blue).
- Measured flow vectors (green for inliers, red for outliers).
- Duck pixel masks and/or bounding boxes.
- The 3σ confidence ellipse for the expected location of each landmark in the image. The confidence ellipses should be blue for successfully tracked landmarks and red for landmarks that are visible, but were not detected.

The right pane (3D view) should display the following:

- The 3σ confidence ellipsoid for the expected camera position.
- The camera frustum for the expected camera position.
- 3σ confidence ellipsoids for the expected landmark positions. The confidence ellipsoids should be blue for landmarks that are visible and successfully tracked, red for landmarks that are visible, but were not detected and yellow for landmarks that are not visible.
- The view should be chosen to ensure the landmarks and the camera location are visible in each frame⁶.

Terminal interface

Examples of valid terminal commands and the expected operation are given below:

Terminal

```
nerd@basement:~/MCHA4400/a2/build$ ./a2 --scenario=6 --interactive=0 --export ../data/indoor/duck.MOV
nerd@basement:~/MCHA4400/a2/build$ ./a2 -s=6 -i=0 -e ../data/indoor/duck.MOV
nerd@basement:~/MCHA4400/a2/build$ ./a2 -s=6 -e ../data/indoor/duck.MOV
Load camera calibration data from ../data/indoor/camera.xml
Run visual navigation for scenario 6 (duck) using video frames from ../data/indoor/duck.MOV
Render the visualisation for each frame without delay and terminate when finished
Export visualisation to video ../out/duck_out.MOV
```

⁶The view may be fixed or move smoothly throughout the video sequence

Terminal

```
nerd@basement:~/MCHA4400/a2/build$ ./a2 --scenario=6 --interactive=0 ../data/indoor/duck.MOV
nerd@basement:~/MCHA4400/a2/build$ ./a2 -s=6 -i=0 ../data/indoor/duck.MOV
nerd@basement:~/MCHA4400/a2/build$ ./a2 -s=6 ../data/indoor/duck.MOV
Load camera calibration data from ../data/indoor/camera.xml
Run visual navigation for scenario 6 (duck) using video frames from ../data/indoor/duck.MOV
Render the visualisation for each frame without delay and terminate when finished
No video is exported
```

Terminal

```
nerd@basement:~/MCHA4400/a2/build$ ./a2 --scenario=6 --interactive=1 ../data/indoor/duck.MOV
nerd@basement:~/MCHA4400/a2/build$ ./a2 -s=6 -i=1 ../data/indoor/duck.MOV
Load camera calibration data from ../data/indoor/camera.xml
Run visual navigation for scenario 6 (duck) using video frames from ../data/indoor/duck.MOV
Render the visualisation for each frame, but pause on the last frame and enable the mouse interactor
No video is exported
```

Terminal

```
nerd@basement:~/MCHA4400/a2/build$ ./a2 --scenario=6 --interactive=2 ../data/indoor/duck.MOV
nerd@basement:~/MCHA4400/a2/build$ ./a2 -s=6 -i=2 ../data/indoor/duck.MOV
Load camera calibration data from ../data/indoor/camera.xml
Run visual navigation for scenario 6 (duck) using video frames from ../data/indoor/duck.MOV
Render the visualisation for each frame, but pause on every frame and enable the mouse interactor
No video is exported
```

Code submission

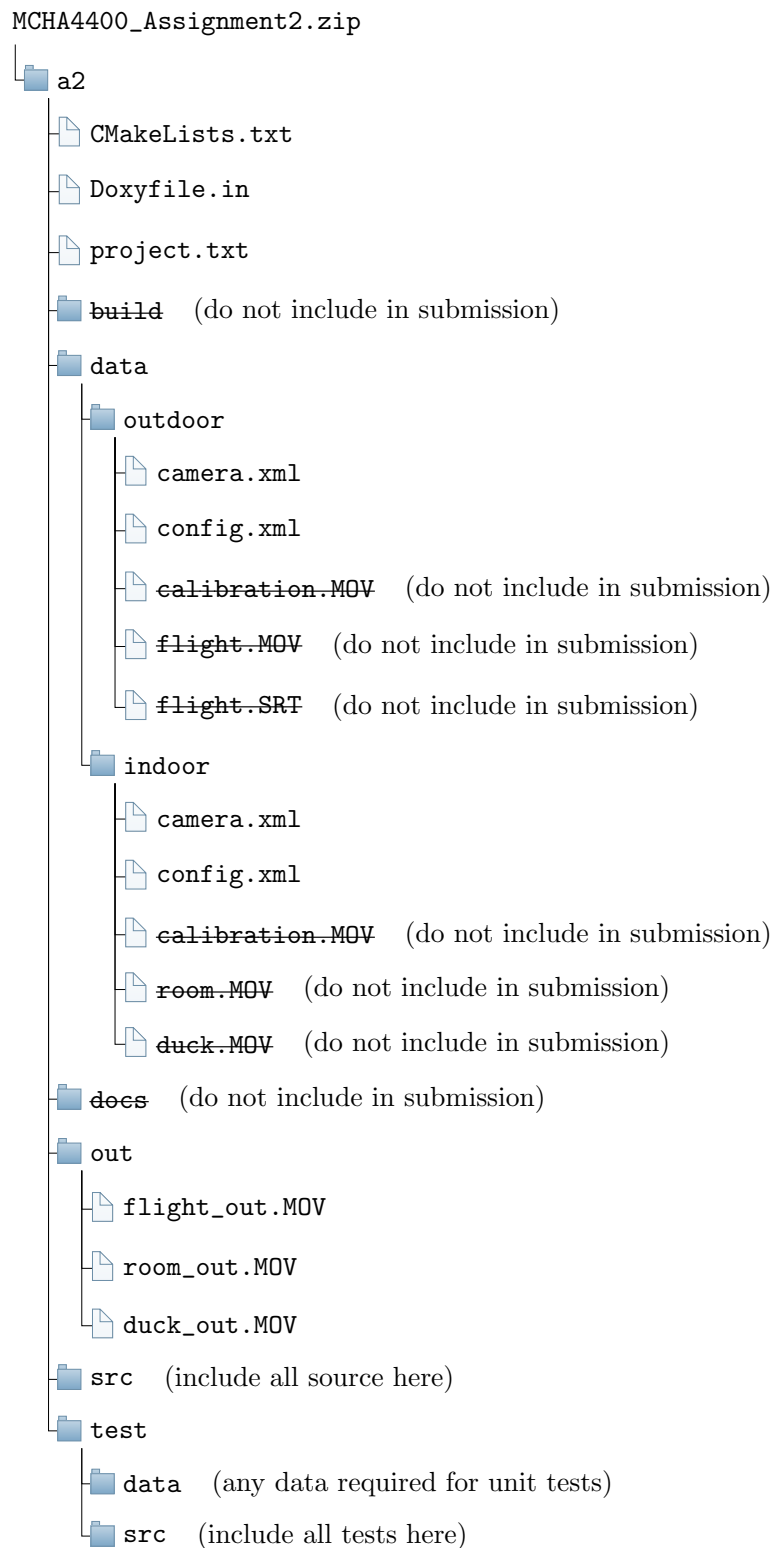


Figure 1: Expected directory structure of submitted ZIP file.

**Before you submit your solution**

Be sure to submit all source files necessary to build and run your solution, except for the input data and temporary build files (see Figure 1). Include the output videos rendered from each scenario.

Your solution is expected to build under the environments detailed in Lab 1. If your solution relies on additional third-party dependencies, **consult with staff before submitting**, so that we can advise on avoiding those dependencies or so we can configure our build environment to support them.

Note:

- Canvas will rename all your files that you submit when they are downloaded for grading, so place all your files within a ZIP archive, so they unpack correctly with the right filenames. Files submitted using other compression formats (e.g., RAR, 7z, etc.) will not be accepted.
- Make sure you click the **submit** button by the due time. If you only **upload**, but don't **submit**, the markers cannot access your files.
- After submitting your solution, **download** it from Canvas before the due time to confirm that you have uploaded a ZIP file that contains the correct files.

Before the due date and time, submit your solution as a single ZIP archive using the assignment submission link on Canvas. Submissions outside of Canvas (e.g., via email or Slack) will not be accepted.